School of Future Tech

Case Study Report

on

Digital Notice Board System (C++)

by

Shankar Mupanna

150096725101

Index

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. Problem Statement / Case Study Design
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study
5. Problem Statement / Case Study Implementation Details and Snapshots
6. Problem Statement / Case Study Results and Conclusion
7. References

---

1. Introduction to the Case Study

Information sharing is an essential part of organizations such as schools, colleges, and workplaces. Notice boards are commonly used to display announcements like exam schedules, events, holidays, and other important updates. Traditional notice boards require manual updates and physical display of information, which may not always be efficient or easily manageable.

This case study focuses on designing and implementing a Digital Notice Board System using C++. The system allows users to add, delete, and view notices dynamically through a menu-driven console application. The program demonstrates how programming concepts such as classes, arrays, loops, conditional statements, and functions can be used to simulate a simple information management system.

The project also introduces the concept of organizing data using object-oriented programming (OOP) techniques, making the system easier to manage and extend in the future.

2. Problem Statement / Case Background (Abstract)

Background

In many organizations, notices need to be updated frequently. Traditional notice boards require manual effort and can become cluttered when many notices are posted. A digital system can manage announcements more efficiently by allowing users to store, update, and display notices dynamically.

Abstract

This case study presents the design and implementation of a Digital Notice Board System using C++. The program provides a menu-driven interface that allows the user to perform operations such as adding new notices, deleting existing notices, and viewing all available notices.

Each notice contains a title and description, which are stored using a class-based data structure. The notices are maintained in an array of objects, allowing multiple notices to be managed simultaneously during program execution. The system demonstrates fundamental programming concepts such as data structures, object-oriented design, and dynamic content management in a simple console-based environment.

3. Case Study Design

The Digital Notice Board System is designed as a menu-driven application that manages notices using structured programming and object-oriented concepts.

The system consists of the following components:

a. Notice Data Structure

Each notice contains the following information:

- Notice Title
- Notice Description

These attributes are grouped together using a class called Notice.

b. Main System Operations

<u>The program provides the following operations:</u>

i) Add Notice
Allows the user to enter a new notice with a title and description and store it in the notice board.

ii) Delete Notice
Allows the user to remove an existing notice by specifying its notice number.

iii) View Notices
Displays all available notices in a structured format.

iv) Exit
Ends the program execution.

c. Program Flow

The program continuously displays a menu and waits for user input. Based on the selected option, the corresponding operation is executed.

<u>Program Flow Structure:</u>

Start
↓
Display Menu
↓
User selects operation
↓
Perform selected operation

↓
Return to menu until Exit is chosen

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

The Digital Notice Board System applies several fundamental programming techniques.

Data Organization using Classes

A class named Notice is used to represent each notice. The class contains data members such as title and description along with functions to input and display notice details.

Array of Objects

An array is used to store multiple notice objects.

Example concept:

Notice notices[100]

This allows the system to manage multiple notices efficiently.

Menu Driven Algorithm

The program uses a do-while loop to repeatedly display the menu and perform operations until the user selects Exit.

Adding a Notice Algorithm

1. Check if the notice board has space available.

2. Accept notice title and description from the user.

3. Store the notice in the array.

4. Increase the notice counter.

Deleting a Notice Algorithm

1. Ask the user for the notice number to delete.

2. Validate the input.

3. Shift remaining notices in the array to fill the empty space.

4. Reduce the notice counter.

Display Notices Algorithm

1. Check if any notices exist.

2. Loop through the notice array.

3. Print each notice title and description.

Technology Stack Used

Programming Language
C++

Concepts Used

- Classes and Objects
- Arrays
- Functions
- Loops

- Conditional Statements
- Menu-driven programming

Development Environment

- C++ compiler (GCC / MinGW / g++)
- IDE such as VS Code, CodeBlocks, or Dev-C++

5. Problem Statement / Case Study Implementation Details and Snapshots

The system is implemented as a single C++ program that includes the following main components.

Notice Class

The Notice class defines the structure of each notice and contains functions for input and display.

Example concept:

```
class Notice
{
  string title;
  string description;
}
```

Main Program

The main program contains:

- Array of Notice objects
- Menu system
- Logic for adding, deleting, and viewing notices

Key Program Modules

Notice Input Module

Accepts title and description from the user and stores them in the notice object.

Notice Display Module

Displays all stored notices in a formatted layout.

Notice Deletion Module

Removes a notice and shifts remaining notices to maintain proper indexing.

Main Program Code

final_mini_project > main.cpp > Notice > display()

```cpp
#include <iostream>
#include <string>
using namespace std;

class Notice
{
public:
    string title;
    string description;

    void inputNotice()
    {
        cout << "Enter Notice Title: ";
        getline(cin, title);

        cout << "Enter Notice Description: ";
        getline(cin, description);
    }

    void display()
    {
        cout << "Title: " << title << endl;
        cout << "Description: " << description << endl;
    }
};

int main()
{
    Notice notices[100];
    int noticeCount = 0;
    int choice;

    do
    {
        cout << "\n===== DIGITAL NOTICE BOARD =====\n";
        cout << "1. Add Notice\n";
        cout << "2. Delete Notice\n";
        cout << "3. View Notices\n";
        cout << "4. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
        cin.ignore();

        switch(choice)
        {
```

0 ⚠ 0

final_mini_project > main.cpp > Notice > display()

```cpp
        switch(choice)
        {
            case 1:
            {
                if(noticeCount >= 100)
                {
                    cout << "Notice board is full!\n";
                    break;
                }

                notices[noticeCount].inputNotice();
                noticeCount++;

                cout << "New Notice Added Successfully!\n";
                break;
            }

            case 2:
            {
                if(noticeCount == 0)
                {
                    cout << "No notices to delete.\n";
                    break;
                }

                int num;
                cout << "Enter notice number to delete: ";
                cin >> num;

                if(num < 1 || num > noticeCount)
                {
                    cout << "Invalid notice number.\n";
                    break;
                }

                for(int i = num-1; i < noticeCount-1; i++)
                {
                    notices[i] = notices[i+1];
                }

                noticeCount--;
                cout << "Notice deleted successfully.\n";
                break;
            }
```

0 ⚠ 0

```cpp
                    notices[i] = notices[i+1];
                }

                noticeCount--;
                cout << "Notice deleted successfully.\n";
                break;
            }

            case 3:
            {
                if(noticeCount == 0)
                {
                    cout << "No notices available.\n";
                    break;
                }

                cout << "\n------ DIGITAL NOTICE BOARD ------\n";

                cout << "Total Notices: " << noticeCount << endl;

                for(int i = 0; i < noticeCount; i++)
                {
                    cout << "\nNotice " << i+1 << endl;
                    notices[i].display();
                }

                cout << "----------------------------------\n";
                break;
            }

            case 4:
            {
                cout << "Exiting program...\n";
                break;
            }

            default:
                cout << "Invalid choice.\n";
        }

    } while(choice != 4);

    return 0;
}
```

0 ⚠ 0

Example Program Output

```
cd "/Users/shankarmupanna/Desktop/c++_codes/final_mini_project/" && g
_codes/final_mini_project/"main
shankarmupanna@Shankars-MacBook-Air c++_codes % cd "/Users/shankarmup
pp -o main && "/Users/shankarmupanna/Desktop/c++_codes/final_mini
_project/"main

===== DIGITAL NOTICE BOARD =====
1. Add Notice
2. Delete Notice
3. View Notices
4. Exit
Enter choice: 1
Enter Notice Title:  Exam Schedule
Enter Notice Description: Exams start from Monday
New Notice Added Successfully!

===== DIGITAL NOTICE BOARD =====
1. Add Notice
2. Delete Notice
3. View Notices
4. Exit
Enter choice: 1
Enter Notice Title: Exam Schedule
Enter Notice Description: Holiday
New Notice Added Successfully!

===== DIGITAL NOTICE BOARD =====
1. Add Notice
2. Delete Notice
3. View Notices
4. Exit
Enter choice: 1
Enter Notice Title: Holiday
Enter Notice Description: College closed Friday
New Notice Added Successfully!

===== DIGITAL NOTICE BOARD =====
1. Add Notice
```

0 ⚠ 0 Ln 95, Col 18

```
shankarmupanna@Shankars-MacBook-Air c++_codes % cd "/Users/shankarmu
pp -o main && "/Users/shankarmupanna/Desktop/c++_codes/final_mini
_project/"main
Enter Notice Title: Holiday
Enter Notice Description: College closed Friday
New Notice Added Successfully!

===== DIGITAL NOTICE BOARD =====
1. Add Notice
2. Delete Notice
3. View Notices
4. Exit
Enter choice: 3

------ DIGITAL NOTICE BOARD ------
Total Notices: 3

Notice 1
Title:  Exam Schedule
Description: Exams start from Monday

Notice 2
Title: Exam Schedule
Description: Holiday

Notice 3
Title: Holiday
Description: College closed Friday
----------------------------------

===== DIGITAL NOTICE BOARD =====
1. Add Notice
2. Delete Notice
3. View Notices
4. Exit
Enter choice: 4
Exiting program...
shankarmupanna@Shankars-MacBook-Air final_mini_project %
```

0 ⚠ 0 Ln 95, Col 18

6. Problem Statement / Case Study Results and Conclusion

Findings

The Digital Notice Board System successfully demonstrates how programming can be used to manage information dynamically. By using arrays and classes, the program efficiently stores and organizes multiple notices.

The menu-driven interface makes it easy for users to interact with the system and perform operations such as adding, deleting, and viewing notices.

Conclusion

This case study demonstrates the design and implementation of a Digital Notice Board System using C++. The system effectively applies core programming concepts including object-oriented design, arrays, loops, and conditional logic.

The project provides a simple yet practical example of how software can simulate real-world information display systems. The system can be further enhanced by adding features such as file storage, graphical interfaces, or network-based notice updates.

7. References

C++ Programming Language documentation and tutorials.

Standard C++ library documentation for strings and input/output operations.

Online resources and tutorials for menu-driven programs in C++.

Books and educational materials related to Object-Oriented Programming and Data Structures in C++.